



PAPER • OPEN ACCESS

Harnessing the power of gradient-based simulations for multi-objective optimization in particle accelerators

To cite this article: Kishansingh Rajput *et al* 2025 *Mach. Learn.: Sci. Technol.* **6** 025018

View the [article online](#) for updates and enhancements.

You may also like

- [Hyperparameter optimisation in deep learning from ensemble methods: applications to proton structure](#)
Juan Cruz-Martinez, Aron Jansen, Gijs van Oord et al.
- [Enhancing friction stir-based techniques with machine learning: a comprehensive review](#)
Noah E El-Zathry, Stephen Akinlabi, Wai Lok Woo et al.
- [Tensor tree learns hidden relational structures in data to construct generative models](#)
Kenji Harada, Tsuyoshi Okubo and Naoki Kawashima



PAPER

OPEN ACCESS

RECEIVED
9 November 2024REVISED
22 January 2025ACCEPTED FOR PUBLICATION
18 March 2025PUBLISHED
16 April 2025

Original Content from
this work may be used
under the terms of the
[Creative Commons
Attribution 4.0 licence](#).

Any further distribution
of this work must
maintain attribution to
the author(s) and the title
of the work, journal
citation and DOI.



Harnessing the power of gradient-based simulations for multi-objective optimization in particle accelerators

Kishansingh Rajput^{1,2,*} , Malachi Schram^{1,3} , Auralee Edelen⁴, Jonathan Colen^{5,6} , Armen Kasparian¹, Ryan Roussel⁴, Adam Carpenter¹, He Zhang¹ and Jay Benesch¹

¹ Thomas Jefferson National Accelerator Facility, Newport News, VA 23606, United States of America

² Department of Computer Science, University of Houston, Houston, TX 77204, United States of America

³ Department of Computer Science, Old Dominion University, Norfolk, VA 23529, United States of America

⁴ SLAC National Laboratory, Menlo Park, CA 94025, United States of America

⁵ Joint Institute on Advanced Computing for Environmental Studies, Old Dominion University, Norfolk, VA 23539, United States of America

⁶ Hampton Roads Biomedical Research Consortium, Portsmouth, VA 23703, United States of America

* Author to whom any correspondence should be addressed.

E-mail: kishan@jlab.org

Keywords: reinforcement learning, Bayesian optimization, genetic algorithm, multi objective, MORL, MOGA, differentiable simulation

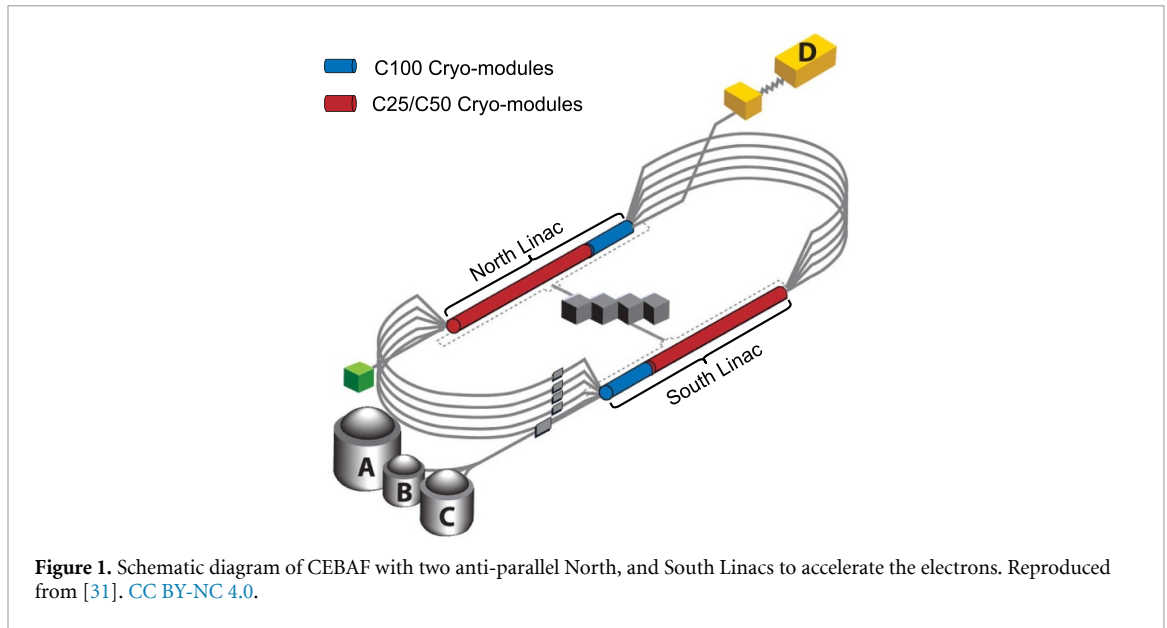
Abstract

Particle accelerator operation requires simultaneous optimization of multiple objectives. Multi-objective optimization (MOO) is particularly challenging due to trade-offs between the objectives. Evolutionary algorithms, such as genetic algorithms (GAs), have been leveraged for many optimization problems, however, they do not apply to complex control problems by design. This paper demonstrates the power of differentiability for solving MOO problems in particle accelerators using a deep differentiable reinforcement learning (DDRL) algorithm. We compare the DDRL algorithm with model-free reinforcement learning (MFRL), GA, and Bayesian optimization (BO) for simultaneous optimization of heat load and trip rates in the continuous electron beam accelerator facility. The underlying problem enforces strict constraints on both individual states and actions as well as cumulative (global) constraints on energy requirements of the beam. Using historical accelerator data, we develop a physics-based surrogate model which is differentiable and allows for back-propagation of gradients. The results are evaluated in the form of a Pareto-front with two objectives. We show that the DDRL outperforms MFRL, BO, and GA on high dimensional problems.

1. Introduction

Particle accelerators are intricate, high-energy machines comprised of numerous specialized components. To ensure efficient operation and precise control, accelerator operators must meticulously adjust multiple component settings to fine-tune the machine.

Jefferson Laboratory's primary particle accelerator, the continuous electron beam accelerator facility (CEBAF) [1], comprises two anti-parallel superconducting radiofrequency (SRF) linear accelerators (linacs) which accelerate electrons. A schematic diagram of CEBAF is shown in figure 1. Each linac contains 25 cryomodules comprising 200 SRF cavities. These cavities are kept at 2 K temperature to maintain superconductivity. The cryomodules are filled with liquid helium regulated by a Central Helium Liquefier (CHL). Each cavity is individually controlled and each has its own unique operating characteristics. An ever-present challenge faced by operations staff is determining the best way to distribute radio frequency (RF) gradients across these cavities to meet the experiment's required energy gain and simultaneously minimize negative impacts to CEBAF and experiments. Two aspects of SRF operations that are directly controlled through the gradient distribution is the heat load imposed upon the cryogenics system through RF operations and the number of Fast Shut Down (FSD) trips initiated by the RF system. Optimizing the RF heat load and FSD trip rate helps lower the wear and tear in CHL and reduce machine downtime. This is a multi-objective optimization (MOO) problem, in which there is a tradeoff between the heat load and the



number of FSD trips. In MOO, the full optimal set of tradeoffs is known as the Pareto-optimal front (or Pareto front). This front defines the best quality solution that can be achieved for a given objective without reducing the quality of a competing objective.

The problem we are considering thus becomes a question of how to efficiently find the Pareto-optimal front defining the tradeoff between the FSD trips and RF heat load, ideally in a way that will translate from offline system analysis to real-world facility operation. In addition to minimizing RF heat load and FSD trips, the gradient distribution needs to produce an energy within a very small tolerance for proper steering of the beam. This introduces a global hard constraint on the optimal solution sets, making the problem very challenging especially when considering the large number of cavities in a linac. From an operational point of view, a quickly-converging algorithm on this high-dimensional MOO problem is desired.

In this paper, we use a surrogate model of the CEBAF RF heat load and trip rates that consists of a series of fits to years of measured data using known exponential functions for each cavity, to train and compare model-free reinforcement learning (MFRL) based conditional multi-objective twin delayed deep deterministic policy gradient (CMO-TD3) [2], non-dominated sorting genetic algorithm II (NSGA-II) [3], multi-objective Bayesian optimization (MOBO) [4], and conditional multi-objective deep differentiable reinforcement learning (CMO-DDRL) [5] algorithms in the offline MOO task described above. We investigate the performance of these algorithms in terms of time- and sample- efficiency, as well as solution quality. We also assess how the performance changes with problem dimensionality (i.e. number of RF cavities used). Finally, embedded in this investigation, we quantitatively assess the impact of using a differentiable system model (as is done in DDRL), in contrast to algorithms that do not use such a model (NSGA-II, TD3, MOBO). We also highlight some advantages the DDRL approach has in terms of being able to actively scan the Pareto-optimal front in a control setting. This investigation highlights the importance of sample acquisition speed in determining appropriate algorithms to use in MOO. It also provides a useful case-study for the accelerator community, which will aid researchers in determining approaches to use for problems on other systems.

The paper is organized as follows: section 2 describes the latest work in machine learning (ML)-based optimization for particle accelerators, section 3 describes the CEBAF optimization challenge, and the methods and results are described in sections 4 and 5 respectively. We close the discussion with some insights and future outlook in section 6.

2. Previous work

ML has been instrumental in solving complex challenges in many science and engineering applications. ML techniques have been applied to a variety of particle accelerator applications including anomaly detection and prediction [6–10], surrogate modeling [11, 12], and beam tuning [13, 14].

In the context of determining Pareto-optimal tradeoffs between competing objectives (and particularly in an offline or design setting), genetic algorithms (GAs) have historically been the algorithm of choice in the accelerator community. GAs have been explored to assist in gradient distribution in CEBAF linacs [15] to

simultaneously minimize both heat load and trip rates while maintaining required energy gain. Charged particle optics design (CPOpts) is a multi-objective GA (MOGA) framework using NSGA-II [3] to optimize particle transmission and beam spot size for designing CPOpts [16]. Researchers proposed a fast multi-objective software framework using evolutionary algorithms to tackle the complex problem of designing and optimizing particle accelerators, such as the Argonne Wakefield Accelerator facility, to achieve optimal machine parameters and beam dynamics [17]. Researchers have developed a ML-based approach to create fast and accurate surrogate models for high-fidelity physics simulations of charged particle accelerators, enabling significant speedups in design studies and experiment planning [18]. MOGA has been used to optimize dynamic aperture in Swiss Light Source upgrade (SLS 2.0) [19]. However, GAs are purely optimization-focused algorithms as it leverages parallelized evolution and does not allow continuous control for online accelerator operations. GAs also require re-running when there is a change in the linac, such as a cavity going offline.

Bayesian optimization (BO) has been explored for particle accelerator optimization [20–22]. Additionally, MOBO, has been used online to efficiently map out Pareto-optimal tradeoffs between competing objectives [4]. Unlike GAs, MOBO learns an underlying representation of the system behavior. As long as the relevant inputs are provided to MOBO, adjustments such as a cavity going offline can be accounted for with approaches such as contextual BO. While there are also avenues such as adaptive BO to account for changing conditions over time [23], BO is primarily aimed at episodic optimization rather than continuous control, and becomes more computationally expensive as the number of sample points increases. A novel MOBO scheme was introduced to efficiently optimize particle accelerator performance online, reducing the number of observations needed to converge by at least an order of magnitude compared to current methods [4]. A MOBO approach has been used to optimize electron beam properties in the SLAC MeV-UED facility, allowing for faster and more efficient online beam tuning by searching the parameter space to identify the best trade-offs between key beam properties [24]. BO has also been explored for fast and safe parameter tuning of swissfel [25], and optimization of laser-plasma accelerators assisted by reduced physical models [26].

In contrast, reinforcement learning (RL) is intrinsically a controls algorithm which can continuously learn and adapt with changing conditions while providing a real-time control [27]. There are several studies using single objective RL for accelerator applications [27–30], however, to the best of our knowledge there are no studies using multi-objective RL (MORL) for particle accelerator optimization or controls.

3. Problem description

CEBAF uses SRF cavities to accelerate electrons to a desired energy. When a RF field is applied, SRF cavities dissipate heat H_i into the cryogenic system. This heat dissipation can be approximated by equation (1). It is characterized by the cavity's current RF gradient G_i , its length l_i , impedance ω_i , and the quality factor $Q_i(G_i)$ [15]. Given the non-linear nature of heat dissipation, poor choices in gradient distribution can result in considerably higher heat loads and reduced cryogenic system stability,

$$H_i = \frac{G_i^2 l_i}{\omega_i Q_i(G_i)}. \quad (1)$$

The length of cavities l_i , and their impedance ω_i are known and standardized to a high precision. The RF gradients G_i are actively controlled during operation. The quality factor Q_i is a function of RF gradient. It is only measured through time consuming procedures that would interrupt beam delivery. As such, quality factors for each cavity are determined only at cryo-module commissioning and through infrequent, ad hoc beam studies. The quality factor can be negatively impacted after commissioning through a variety of means, including the introduction of particulate contamination or frozen gases. How to parasitically (without beam intervention) determine the changes in the quality factor is an open question complicated by the fact that the cavities in a cryogenic module share a single cryogenic Helium supply and associated control system. As such, we use a fixed Q_i value per cavity equal to that used in operations.

Legacy cryogenic modules from CEBAF's 6 GeV era are still in operation and make up the majority of cryogenic modules in linacs. These are typically referred as C25 modules due to their median energy gain of 25 MeV per cryogenic module. One prevalent issue with C25s is that arc faults regularly occur as field emission from cavity walls charges its surface during RF operations and eventually discharges. When an arc fault is detected, the RF system for that cavity is turned off and the electron beam production is stopped to prevent potential damages.

Many C25s were refurbished into C50s in order to increase their median energy gain and mitigate this failure mode. For the surrogate model used in this study, the following family of log-linear statistical models,

Table 1. Optimization environments; their state, and action dimensions, energy constraint, and reference points, and ideal points (lower bound) for hypervolume calculation.

Env	Number of cavities	$E_{\text{linac}} \pm \delta E$ (MeV)	Hypervolume	
			Ref (H, T)	Ideal (H, T)
8D	8	20.08 ± 0.40	(22.4, 0.05)	(20.9, 0.015)
16D	16	50.00 ± 0.60	(100.0, 0.40)	(88.0, 0.015)
32D	32	120.00 ± 0.80	(290.0, 0.40)	(262.0, 0.010)
North linac	200	1050.00 ± 2.00	(2530.0, 6.00)	(2380.0, 1.000)

as described in equation (2), were developed using historical data that related a cavity's operating RF gradient to the arc fault trip rate. Here, T_i is the trip rate, G_i is the cavity RF gradient, F_i is the fault gradient, and A and B_i are regression parameters. F_i and B_i are fit from the data and A is set to -10.268 based on a previous study [32],

$$T_i = e^{A+B_i(G_i-F_i)}. \quad (2)$$

In addition, the total energy gain (defined in equation (3)) across the linac needs to be within a small tolerance of a target value ($E_{\text{linac}} \pm \delta E$) in order to maintain proper steering and experimental requirements. With the energy constraint, if one or more gradients are decreased, other cavities need to compensate by increasing their gradients. As each cavity has their own characteristic curves for heat load and trip rates, it introduces a trade-off between two objectives in an optimal solution set. Finally, the gradient distribution must account for the unique operating characteristics of each cavity. For example, hardware failures may lock cavities to a specific gradient, and other limitations may restrict the range of RF gradients a cavity can sustain. These issues typically fall neatly into a distribution strategy as they are captured in a cavity's upper gradient limit,

$$E = \sum_{i=0}^N l_i \cdot G_i. \quad (3)$$

From the above problem description, we formulate optimization problems using a physics based surrogate to model the RF heat load and FSD trip rate based on the gradient distribution. Equation (4) describes the optimization problem. The optimization environment uses the OpenAI gymnasium framework [33], allowing seamless integration with different algorithms and packages,

$$\begin{aligned}
 \text{Minimize} \quad & H(\mathbf{G}) = \sum_{i=1}^{N_c} H_i(G_i), \quad T(\mathbf{G}) = \sum_{i=1}^{N_c} T_i(G_i) \\
 \text{Subject to} \quad & |E_{\text{linac}} - \sum_{i=1}^{N_c} G_i l_i| < \delta E, \\
 & a_i \leq G_i \leq b_i.
 \end{aligned} \quad (4)$$

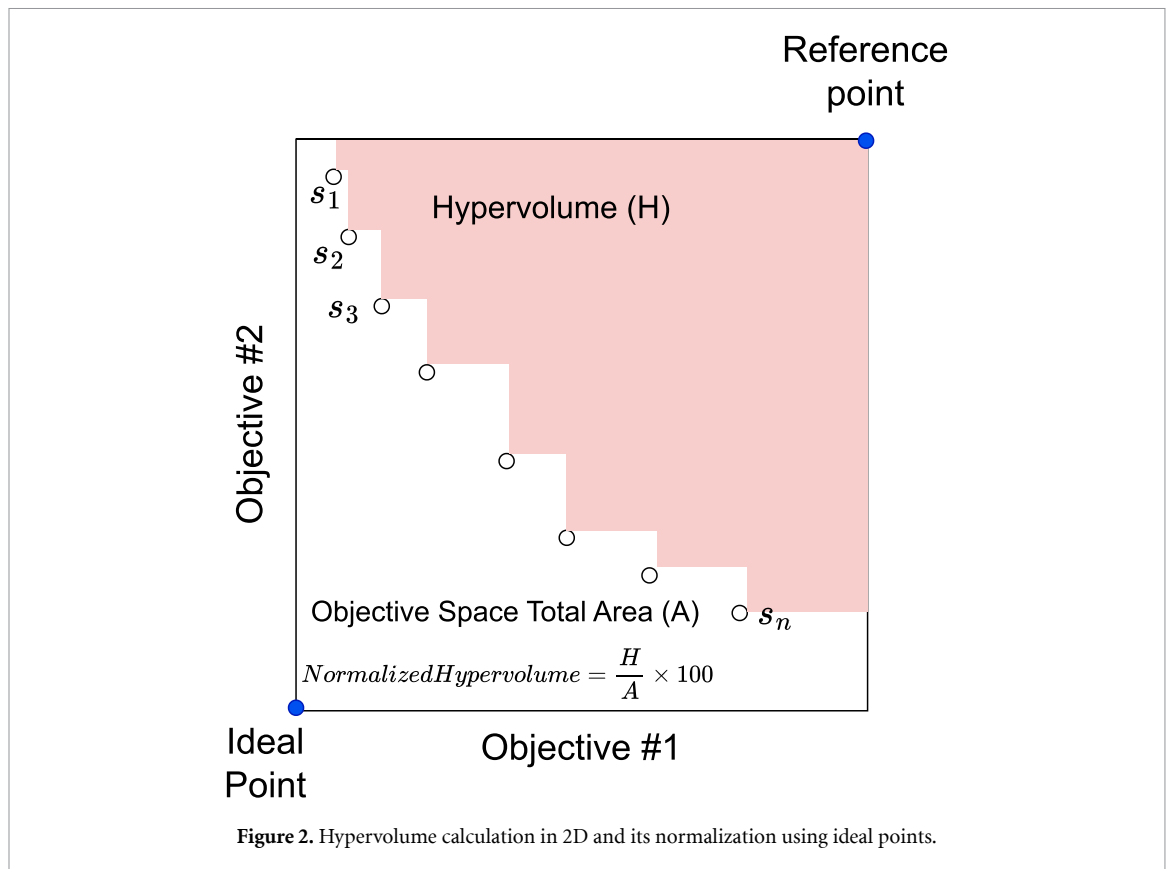
Here $H(G)$ and $T(G)$ are total RF heat load and FSD trip rate, a_i , b_i are the lower and upper gradient limits respectively defined by the operational constraints, and N_c is the number of cavities under consideration.

We formulate four optimization environments to gradually increase the problem complexity. The four environments contain 8 (single cryomodule), 16 (two cryomodules), 32 (four cryomodules), and 200 (entire north linac) RF cavities respectively for simultaneous optimization of heat load and trip rate. We use operational limits on individual cavity gradients as hard constraints. The energy gain requirements are described in table 1 where the fraction of valid energy range is 4.9%, 2.7%, 1.5%, and 0.5% of the energy range of total searchable space in 8D, 16D, 32D, and north linac problems respectively.

As the environment mainly consists of physics equations and is based on historical data, it can be implemented such that it is differentiable. We implement the environment in Tensorflow [34], which allows automatic differentiation and backpropagation of gradients during DDRL training. We note that other algorithms in this study do not leverage differentiability of the environment.

4. Methods

Solutions to MOO problems are represented as a Pareto front. A Pareto front represents a set of solutions where no objective can be improved without worsening another. In other words, it consists of all the optimal



solutions that provide the best trade-offs among conflicting objectives. The hypervolume (or S-metric) [35] is a measure used to evaluate the quality of a Pareto front. It quantifies the volume of the objective space that is dominated by a given Pareto front relative to a reference point. The reference point is a point in the objective space that is typically worse than the best possible values for all objectives. The hypervolume is calculated by measuring the volume of the region in the objective space that is dominated by the Pareto front and bounded by the reference point. We normalize the hypervolume to percentage using the ideal points (lower bound of objective space). The reference and ideal points for each problem are listed in table 1. We use the pymoo [36] package to compute hypervolumes and monitor the Pareto front coverage during and at the end of evolution/training. Figure 2 demonstrates the hypervolume calculation and normalization. Using this hypervolume metric, we evaluate MOO algorithms for our CEBAF problem. We describe each of these algorithms below.

4.1. GA

GA is a type of optimization technique that mimics the process of natural evolution to find the best solution to a problem. It starts by creating a random initial population of potential solutions, known as individuals. Each individual is evaluated based on its fitness, or how well it solves the problem. In each subsequent generation, the algorithm uses two processes to create new individuals: recombination and mutation. Recombination combines the characteristics of two existing individuals to create a new one, while mutation introduces random changes to an individual's characteristics. The new individuals are then evaluated and compared to the existing population. The algorithm then selects the top half of the individuals with the best fitness values to move on to the next generation, effectively eliminating the less fit individuals. This process is repeated multiple times, allowing the fittest individuals to adapt and improve over time. As the algorithm iterates, it is likely to converge on a better solution, making it a powerful tool for solving complex optimization problems.

In this study, we employ the NSGA-II [3] to tackle a MOO problem. NSGA-II is a fast and elitist GA specifically designed for multi-objective problems, with a computational complexity of $O(MN^2)$ for a problem with M objective functions and a population of size N . When dealing with a population that involves multiple objective functions, individuals can be ranked and ordered using two key attributes: nondomination rank and crowding distance. An individual is considered non-inferior to another with the same nondomination rank, but inferior to one with a lower rank. When individuals have the same rank, they are ordered based on their crowding distance, with higher values indicating lower density in the solution

space. This suggests that we should prioritize preserving individuals with higher crowding distances to maintain a diverse and well-spread population. NSGA-II can be briefly described as follows.

- (i) Create the first generation population randomly but ensure that each solution is within the constraints; sequence them by the nondomination rank.
For a sequenced population of size N , we can generate the new population in the following two steps.
- (ii) Create another N individuals by recombination and mutation of the existing ones.
- (iii) Sequence the $2N$ individuals by rank and discard the N individuals with higher ranks. It is important to note that the constraints are folded into the fitness score. Any solution that violates the constraint(s) is assigned a low fitness score to not let them proceed to the next generation.
Now we obtain the new generation population of size N .
- (iv) Repeat steps 2 and 3 for a predetermined number.

4.2. BO

BO is a black-box method that is traditionally used to optimize expensive, unknown functions [37]. More recently, BO has been used extensively in the context of online optimization of particle accelerators (see [22] for a summary). In BO, a probabilistic model is used to predict the mean and uncertainty of the function to be optimized based on the input variable values. This model is often composed of a Gaussian process (GP), which is appealing because GPs are able to learn approximations of functions in a sample-efficient way (e.g. even from tens of samples) and inherently provide reasonable uncertainty estimates. During optimization, points are sampled and the GP is retrained at each iteration. An acquisition function is then used in conjunction with the GP model to choose a subsequent point to sample, which balances the uncertainty in the prediction with the likelihood of reaching a better optimum. One downside of GPs is that they do not scale as well as alternative models such as neural networks with respect to training data size, resulting in longer inference times that increase as the training dataset grows. This in turn affects the number of variables in a given optimization problem that can be solved in a time-efficient manner with GP-based BO. There are alternative approaches to improve scaling, such as using correlated and physics-informed kernels [21, 38], Bayesian neural networks [39] instead of GPs, and trust-region based BO [40]; an overview can be found in [22].

Here, we use constrained MOBO, which models each individual objective as an independent GP and uses expected hypervolume improvement (EHVI) [41] as the acquisition function.

We also use learned output constraints, in which observed constraints are modeled as separate GPs and sampling is conducted with an acquisition function that balances the likelihood of objective improvement and constraint violation, according to the implementation described in [42]. We use implementations in the Xopt package for MOBO [43] which is based on the implementation of MOBO in Botorch [44].

One modification made to the MOBO algorithm in this case to handle the high dimensionality of the input space, was introducing a heuristic to optimize the acquisition function. Generally, the acquisition function is numerically optimized from a set of random initial starting points. This, however, can be a poor strategy in tightly constrained, high dimensional input spaces since the search space increases exponentially. To address this challenge, we assume that the objective functions are relatively smooth, implying that the EHVI acquisition function will be maximized in a local region near previously observed Pareto-optimal points in input space. We leverage this assumption by initializing the acquisition function optimization at the location of Pareto-optimal points in input space if they are available (otherwise random points are chosen). In empirical tests, this showed a significant reduction in the number of iterations needed to converge to the true Pareto front in the ZDT1 test problem [45].

The returned energy, trip rate, and heat load from the optimization environment are used to assess whether constraints are violated, and the valid samples for trip rate and heat load from all previous interactions are used to calculate the hypervolume of the Pareto front. The constraints are modeled separately from the objective, and both are taken into account when sampling new points.

4.3. RL

RL is a sub-field of ML. The main components of RL are the agent and the environment. The agent is composed of the decision making logic which includes a policy. The environment is the complex system that the agent interacts with. RL agents repeatedly interact with the environment to learn how to make optimal decisions. The environment returns a reward or penalty based on how favorable a given interaction was towards the optimal solution. The environment can be an offline virtual physical, or surrogate model, or the real physical system. In our case, the surrogate model captures the accelerator's behavior when changing the cavity gradients such as the heat load, trip rate, and beam energy.

Algorithm 1. Conditional multi-objective TD3.

```

Initialize critic networks  $Q_{\theta_1}, Q_{\theta_2}$  and policy network  $\pi_{\phi}$ 
Initialize target networks  $\theta'_i \leftarrow \theta_i$  and  $\phi' \leftarrow \phi$ 
Initialize the replay buffer  $\mathcal{B}$ 
for  $t$  in  $1, \dots, T$ , do
    Select action  $a$  using the policy  $\pi_{\phi}(s|\alpha) + \epsilon$ ;  $n \sim \epsilon(t, \sigma)$ ;  $s$  is the current state; and  $\alpha$  is the conditional parameter
    that defines the priority of each objective.
    Observe the next state ( $s'$ ) and associated rewards ( $r$ )
    Store transition tuple  $(s, a, r, s', \alpha)$  in  $\mathcal{B}$ 
    Randomly sample  $N$  transitions from  $\mathcal{B}$ 
     $a' \leftarrow \pi_{\phi'}(s'|\alpha) + n$ ;
     $y_i \leftarrow r + \gamma \min_i Q_{\theta'_i}(s', a')$ 
    Update critics  $\theta_i \leftarrow N^{-1} \sum (y_i - Q_{\theta_i}(s, a))^2$ 
    if  $t \bmod d$  then
        Update conditional policy gradient:
         $\nabla_{\phi} J(\phi) = N^{-1} \sum \nabla_a [\alpha \cdot Q_{\theta_1}(s, a)|_{a=\pi_{\phi'}(s|\alpha)}]$ 
        Update target networks:
         $\theta'_i \leftarrow \tau \theta'_i + (1 - \tau) \theta_i$ 
         $\phi' \leftarrow \tau \phi' + (1 - \tau) \phi$ 
    end
end

```

RL is commonly used for control applications, however, it can be used for optimization for a static environment. In a typical RL setting, the objective of the agent is to maximize the infinite sum of discounted future rewards. However, in this study, we approach the multi-objective problem as a pure optimization problem. Thus the agent is trained to optimize the environment in a single step limiting the exploration to short term (current) reward maximization.

MORL research is pivotal in addressing the complex decision-making processes that require balancing multiple, often conflicting, objectives. For this problem, we define the reward vector based on the two objectives in equation (5). Here the reward vector $[R_h, R_t]$ is normalized between 0 and 1, and the penalty P , is a relatively high value when the energy constraint is violated by a significant margin,

$$R_h = -1 \times \sum_{i=0}^N H_i + P \quad \text{and} \quad R_t = -1 \times \sum_{i=0}^N T_i + P \quad (5)$$

$$\begin{cases} -5 \times |E - E_{\min}| & \text{if } E < E_{\min} \\ -5 \times |E - E_{\max}| & \text{if } E > E_{\max} \\ 0 & \text{Otherwise} \end{cases}$$

Current RL approaches to solve MOO problems are split between single-policy and multi-policy approaches [46]. We focus on a single-policy solution leveraging existing research, such as preference driven MORL [47]. For this paper, we study two implementations of RL described in sections 4.3.1 and 4.3.2.

4.3.1. CMO-TD3

The first implementation is based on the TD3 algorithm [2], with two key differences:

- (i) The policy, $\pi_{\theta}(s|\alpha)$, takes a conditional probability vector (α) defining the priority for each objective
- (ii) The critic predicts a Q-value vector providing the value for each objective separately

As shown in algorithm 1, the policy optimization collapses into a single objective when taking the dot product of the conditional and critic's Q-value vectors. This yields a linear combination of the rewards with the weights of each objective being proportional to the importance of the objective to the end user. Optimizing for a single static combination of objectives causes the agent to find a single point on the Pareto front. To learn the full Pareto front, we vary the conditional probability vector α during training.

4.3.2. CMO-DDRL

Differentiable techniques are very powerful and sample efficient in learning concepts [48], especially compared to traditional RL algorithms that rely on exploration by sampling the environment. In contrast to

Algorithm 2. CMO-DDRL.

```

Initialize actor policy  $\pi_\phi$ 
Initialize the policy learning rate  $l = L_0$ 
for  $t$  in  $1, \dots, T$ , do
    Select action  $a$  using the policy  $\pi_\phi(s|\alpha)$ ; where  $s$  is the current state; and  $\alpha$  defines the priority of each objective.
    Observe the next state ( $s'$ ) and associated reward vector ( $r$ )
    Training Loop;
    Sample  $\alpha \sim \text{Dirichlet}(\alpha_1, \dots, \alpha_k)$ ; where  $k$  = number of objectives
     $a \leftarrow \pi_\phi(s|\alpha)$ 
     $s', r \leftarrow \text{env}(a)$ 
    Update conditional policy gradient:
     $\nabla_\phi J(\phi) = N^{-1} \sum \nabla_a [\alpha \cdot r(a) |_{a=\pi_\phi(s|\alpha)}]$ 
    if  $t \bmod l = 0$  and  $l > l_{\min}$  then
        | Anneal learning rate  $l \leftarrow l/\beta$ ; where  $\beta$  is learning rate reduction factor
    end
end

```

the model-free CMO-TD3 algorithm, DDRL relies on the environment being differentiable and back-propagates the short (or long) term reward through the environment to update the agent weights and biases.

The agent interacts with the environment in a manner analogous to TD3, however it does not require critic model or a buffer for storing experiences. Instead, it can utilize the reward from the environment directly for backpropagation. The reward can be formulated as either short term or long term. As described in algorithm 2, the agent observes the current state of the environment, generates a batch of actions conditioned on conditional probability input α , applies them to the environment to compute the reward (single-step) and next step. Next, the agent employs automatic differentiation to update its weights and biases to optimize the reward. It is important to note that this agent is used for one-step optimization, however, it can be used for sequential control problems modeled as a Markov decision process by unrolling N number of future steps as described in a recent study [5].

5. Results

We evaluate MOGA, MOBO, CMO-TD3, and CMO-DDRL algorithms on the MOO environments described in section 3 and compare their performance in terms of quality of solutions and convergence time. On all the problem environments, we first benchmark the results using MOGA algorithm as described in appendix. To provide a statistically robust analysis, we conducted 16 trials for each algorithm using unique random seeds in each trial. We present the summarized results by computing the 2σ confidence bounds using the corresponding quantiles of 0.023 and 0.977 for the lower and upper bounds, respectively.

We set the convergence criterion as reaching maximum hypervolume for a given pair of algorithm and problem. In addition, we use GA results as benchmark; If an algorithm do not reach within 2% of the hypervolume achieved by GA on a given problem, it is considered not converged. On the smaller scale problems (8, and 16 dimensional), all four algorithms converge to a similar Pareto front. However, as the size of the state and action space is scaled up to 32 and 200 (North linac), CMO-DDRL and MOGA converge to similar optimal solutions while none of the trials for MOBO and CMO-TD3 could converge in a given time span. Figure 3 shows there is no Pareto front for CMO-TD3 and MOBO for North linac optimization as they produce infeasible solutions that are outside the energy constraint and do not converge. To demonstrate the evolution of these algorithms during training, figure 5 shows hypervolume coverage with time, and figure 6 shows the hypervolume coverage per algorithm step. This highlights the differences between the algorithms in terms of both time-efficiency and sample-efficiency.

For MOGA, CMO-DDRL, and CMO-TD3, we record the hypervolume at every 100 steps whereas for MOBO algorithm we record it at every step, due to the smaller number of overall steps taken and greater computation time per step relative to the other algorithms.

All the algorithms were run within a consistent environment for a fair comparison of speed and convergency. All the trials presented in this study were implemented in Python 3.10, were run on a dual 64 core AMD EPYC 7763 (Milan) CPU with 128GB memory with AlmaLinux v9.4. It is important to note that the convergence times shown in figures 4 and 5 only account for algorithm training/evolution time and do not include hypervolume calculation (if not required by the training step) or results saving. In terms of time-efficiency, it is evident that MOGA is faster on our smaller scale problems (16 and 32 -dimensional)

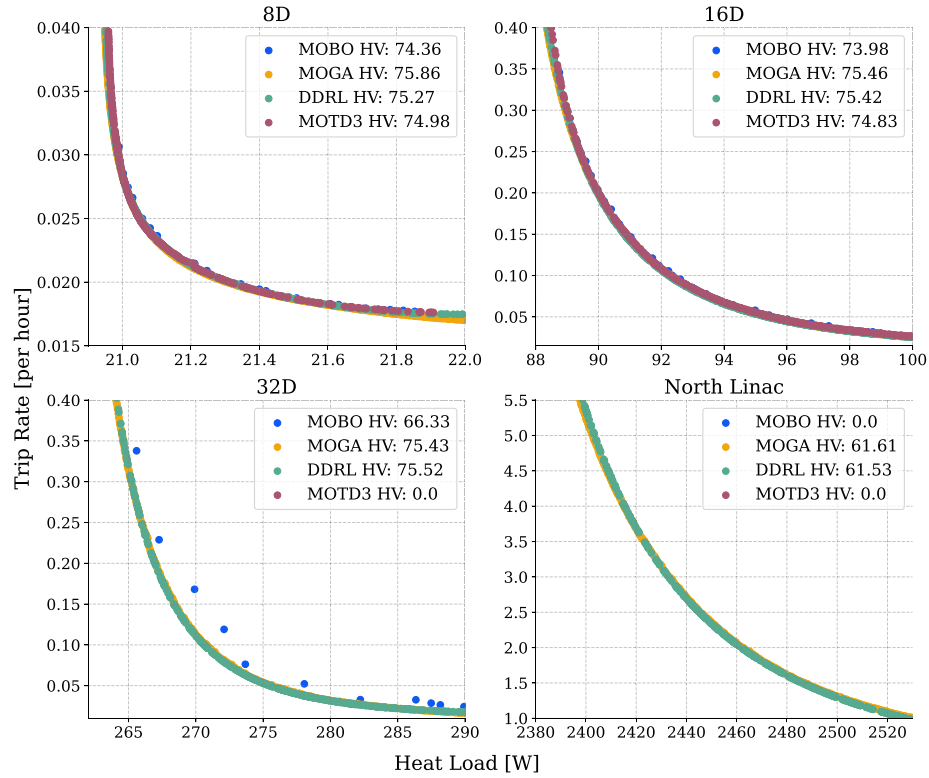


Figure 3. Comparison of optimal Pareto front produced by MOBO, MOGA, MOTD3 and DDRL algorithms. The best Pareto fronts are chosen for each algorithm from 16 trials. Hypervolumes are displayed in the legend.

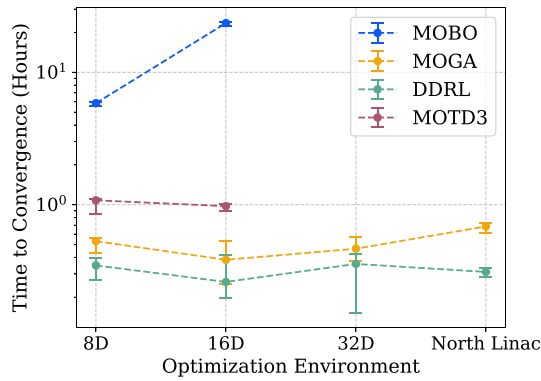


Figure 4. Time taken to converge; Each point represents the median and the error bars denote a 2σ confidence bound over 16 trials. The 32D and North Linac results for MOBO and MOTD3 are not presented, as they do not converge within the 24 h time limit.

than all the other algorithms, where as on the 32D problem the convergence speed of MOGA is comparable to CMO-DDRL, and on the larger scale problem of 200D, CMO-DDRL is fastest to converge.

The MOBO algorithm is slowest to converge on all the optimization environments. This is also evident in figure 4 that displays median convergence time taken by each algorithm with 2σ confidence bounds. Figure 4 also indicate that the run times are roughly constant for highly parallel MOGA, CMO-DDRL, and CMO-TD3 when scaling the problem complexity whereas it grows rapidly for MOBO going from 8D to 16D. All the trials were run for a maximum of 24 h. In case of MOBO on 32D problem it was still evolving at 24 h mark and had not fully converged. As such, it is not included in figure 4.

On the other hand, MOBO takes significantly fewer iterations to converge on the smaller scale problems, as can be seen in figure 6. It also scales poorly with the state and action space size. As we go from the 8-dimensional to the 16-dimensional problem, the time to convergence increases to almost four times. This highlights that in cases where the time cost of acquisitions is high and problem dimensionality is modest, the sample-efficiency of MOBO makes it appealing; however, in cases such as ours where the time cost of

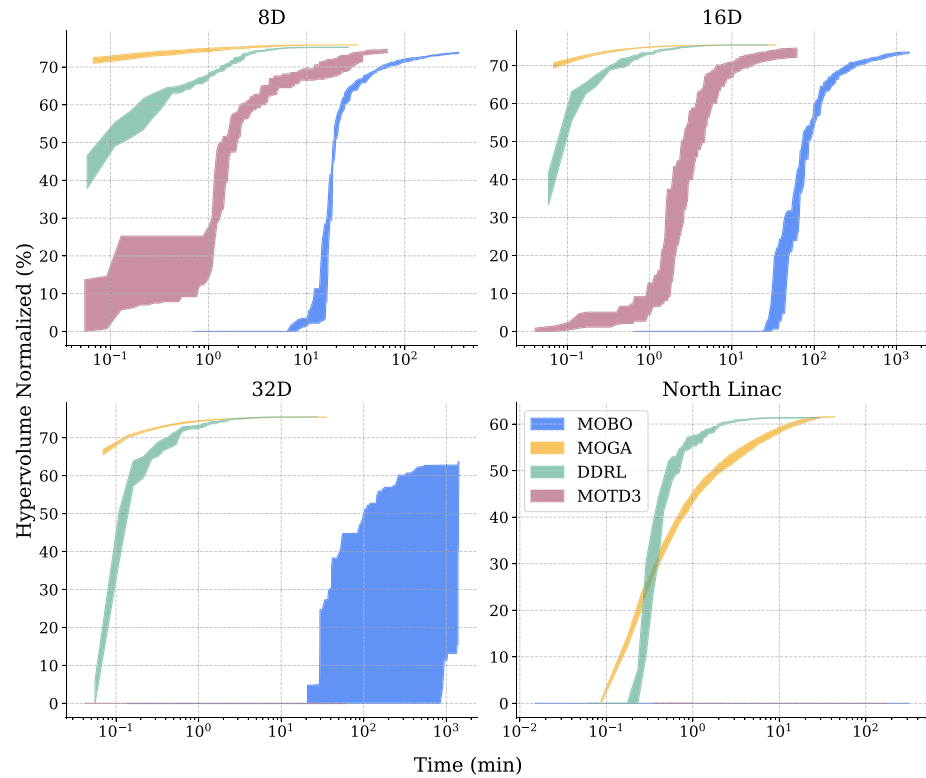


Figure 5. Pareto front coverage over time on different environments with the four algorithms. x -axis (log scale) represents time in minutes. While MOBO takes longest to converge, DDRL is the fastest on the large scale problem (200-dimensional). See also the corresponding convergence plot in terms of samples/iterations in figure 6. Note—the time axis is shown in log scale and does not include hypervolume at 0 s which is zero for all the algorithms due to random initialization.

acquisitions is very low, CMO-DDRL and MOGA provide overall greater time-efficiency despite their lower sample-efficiency. This is an important consideration when choosing an algorithm for other problems.

Notably, even though MOGA converges, it takes many generations to converge compared to some other problems in accelerators (e.g. see [18]), especially for the 200D North linac version of the problem. Observing the MOGA convergence, starting from random sampling the algorithm moves slowly from an initial infeasible region (due to the energy constraint) and eventually reaches the valid region of parameter space. We also note that extensive random sampling of the parameter space for the 200D case does not produce valid samples. This leads us to speculate that one reason some algorithms, such as MOBO and MOTD3, may struggle to converge in this case is that the optimum is a very narrow one in a very large parameter space. MOGA through many parallel samples eventually converges, and DDRL can take advantage of the differentiable model to aid the direction of search. While techniques such as trust region BO can aid optimization of deep, narrow minima [22], in this case we speculate that the high dimension of the parameter space, coupled with starting very far in parameter space from the minimum, make it challenging to converge. Techniques that could potentially improve performance include using priors or a correlated kernel from the differentiable model [21, 49–51], starting closer to the valid region of parameter space (as is often possible when running online), or other approaches to reduce the computational cost of the GP modeling or acquisition function optimization. We plan to study this further in future work.

One of the key challenges for the current implementation of MORL is intrinsic sample inefficiency for the default OpenAI gymnasium sampler when applied to a high-dimensional problem with a global hard constraint. The probability of sampling a valid action that incurs no reward penalty is nearly zero, which forces the agent to expend significant time optimizing in the phase space region outside the target energy constraint.

From these results, we find that DDRL preforms better for large dimensional control applications as it can leverage back-propagation to quickly converge. In contrast MOTD3 is relatively challenging to train in

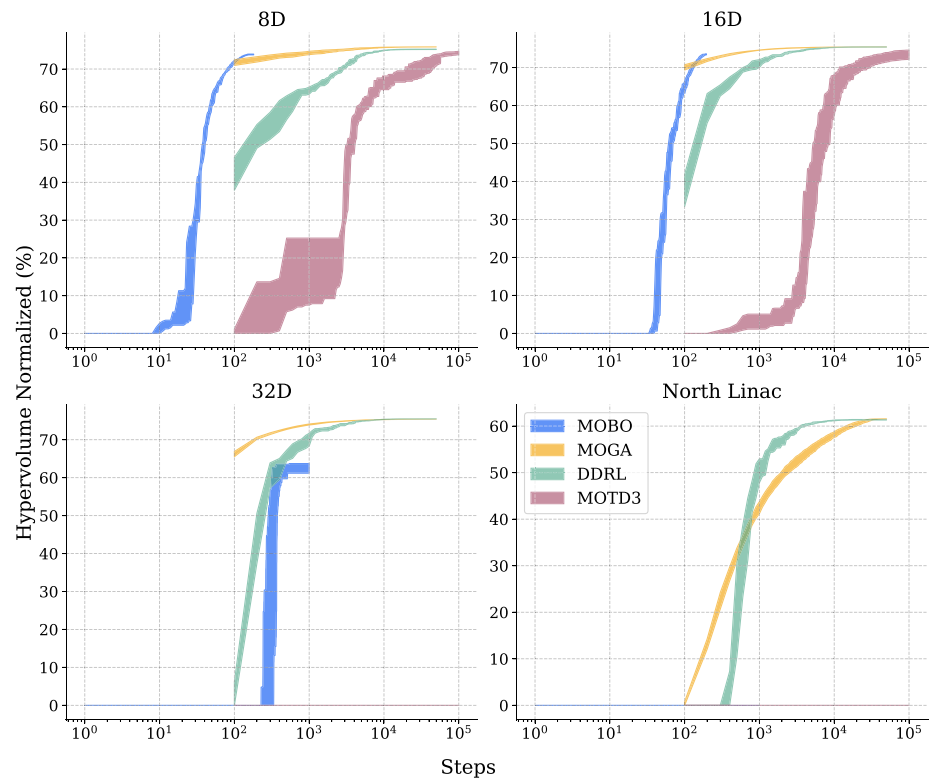


Figure 6. Pareto front coverage on different environments with the four algorithms. Note—the x-axis is shown in log scale and does not include iteration 0, as such the Hypervolume at that step (zero due to random initialization) is not shown in the figures.

this setting due to its reliance on exploration and low sample inefficiency. On the other side, MOGA, and MOBO are suitable for optimization problems. MOBO is comparatively slow in this setting where the environment execution is very fast. Our results suggest it would obtain greater relative performance on environments where the sampling time is slower.

In considering how these algorithms would translate to use on the actual accelerator, as opposed to the offline optimization problem described here, it is important to consider the rate at which settings and read-backs can be made and how close the surrogate model behavior is to the real machine behavior. One reason BO is popular for online optimization is because it can be used in cases where there is no previous data available. If starting from a good operating point on the accelerator, it may not struggle to converge in the same way observed in this broader offline optimization problem. BO also directly takes into account noise and other sources of uncertainty that occur on real-world systems. For DDRL and MOTD3, if the surrogate system is close enough to the real accelerator behavior, the algorithms may be able to be trained offline, as was done here, and transferred to the real system without substantial additional retraining. This prospect would require further study to assess, but the fact that the initial surrogate model used in training is partially data-driven lowers some of the risk in this process. The ability to retrain the RL agent depends on the sampling speed and data availability. On the real system, heat load can be quickly inferred from the cryogenic system (though it may be noisy); in contrast, trip rates are only evaluated theoretically based on historical data and would need to be approximated.

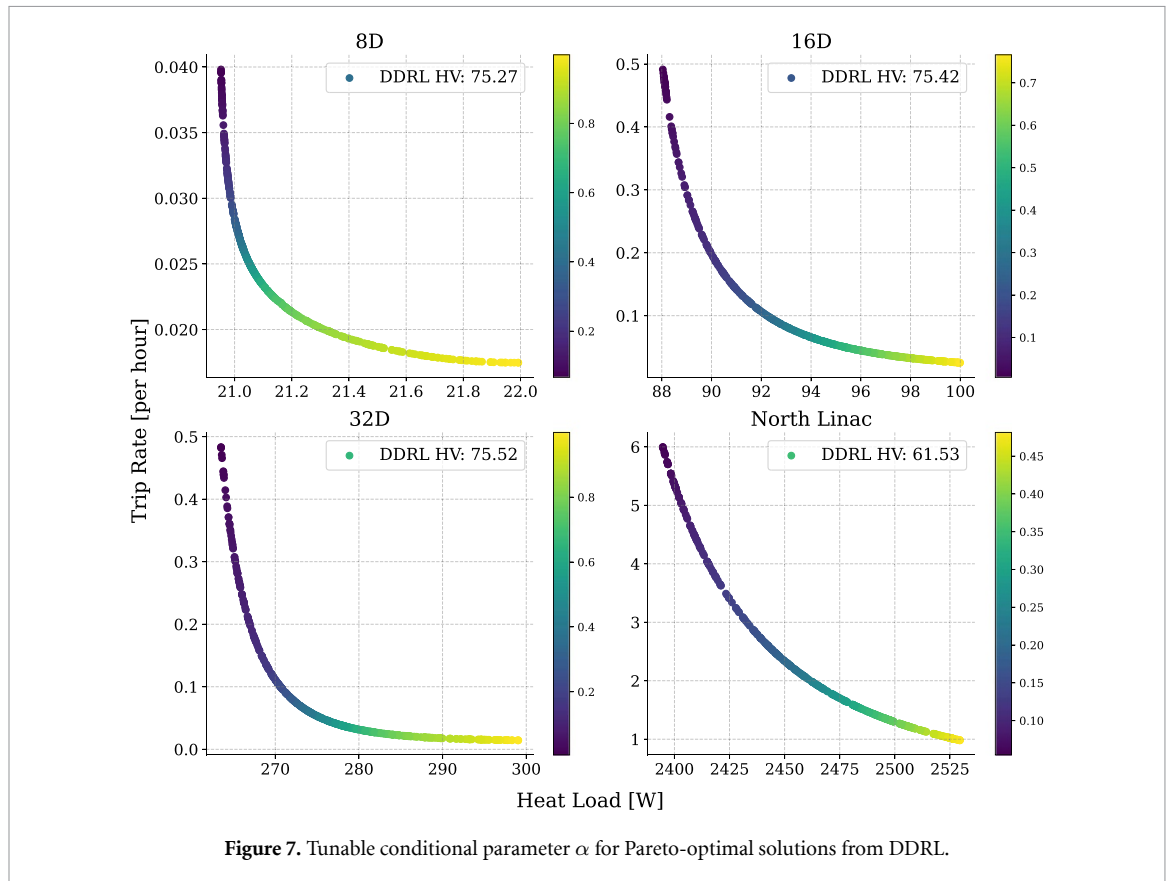
Finally, another major difference between the algorithms is that MOGA uses parallel execution in sample acquisition, which is not possible in a real-world system. In contrast, DDRL, TD3 and MOBO use serial acquisition, meaning these can be used in the real-world optimization/control setting. A summarized version of the advantages and disadvantages are provided in table 2.

5.1. Conditional tuning for multi-objective solutions

Optimal solutions to MOO problems are produced in the form of a Pareto front. It is often required to prioritize one or more objectives over others. This is particularly important for deployed policies where operators may need to focus on different objectives as demanded by operational needs. Conditional input provides a tunable ‘knob’ to the operators where they can change the conditional input to focus on one objective compared to other(s); in our case, with conditional tuning, operators can focus on heat load minimization vs trip rate minimization and vice versa as demanded by the operational needs. Both MOTD3 and DDRL policies use a conditional input to produce different solutions on the optimal Pareto front. The

Table 2. Pros and cons of the algorithms under study. Desired qualities are shown with bold text.

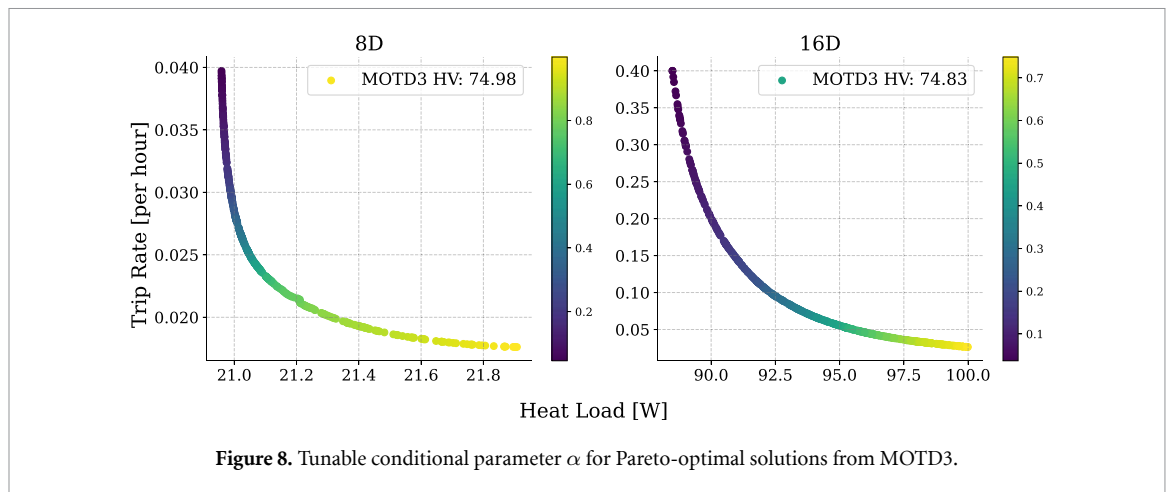
Properties/Algorithm	MOGA	MOBO	MOTD3	DDRL
Computational cost	Low	High	Low	Low
Sample efficiency	Moderate	High	Low	High
Requires differentiable sim	No	No	No	Yes
Scaling with problem size	Moderate	Poor	Moderate	Good
Suitable for continuous control	No	Yes ^a	Yes	Yes^b
Suitable when system model is	Fast or Slow	Slow	Fast	Fast or Slow

^a Only considers current state and action^b With finite number of steps in Markov decision process**Figure 7.** Tunable conditional parameter α for Pareto-optimal solutions from DDRL.

solutions produced by conditional policies are correlated with the optimal solutions on Pareto front. Figure 7 shows the Pareto fronts that the DDRL algorithm produced with the colorbar representing conditional input. It is clear that the distribution of the optimal solutions follow a monotonic behavior with α , making α a tunable knob to get to different solutions on the optimal Pareto front. This behavior is consistent with MOTD3, as shown in figure 8 for the two environments where it converges.

6. Conclusion

In this paper, we compared four algorithms (CMO-DDRL, CMO-TD3, MOBO, and MOGA) on a MOO problem to simultaneously optimize both the RF heat load and FSD trip rate via gradient distribution in CEBAF linacs constrained by an energy requirement. Four optimization problems are formulated to show how this comparison depends on problem complexity, which scales from a single cryomodule to the entire CEBAF North linac. We demonstrated that CMO-DDRL leverages differentiability of the optimization environment to outperform other algorithms in convergence speed on the high dimensional problem. On smaller scale problems, all four algorithms reach similar solutions. However, convergence speed comparison reveals that MOBO is slowest to reach optimal solution even though being sample efficient. It is important to note that MOBO and CMO-TD3 did not converge on the larger scale CEBAF problems, which we suspect is due to the high dimensionality and particular difficulty the global hard constraints in this case. Potential future work can include enhancing CMO-TD3 warmup on large scale problems with better random sampling



that can include hard constraints, and optimizing MOBO in various ways to deal with the high dimensional constrained problem. Another avenue of interesting future research is usage of MORL algorithms for sequential control, especially DDRL and evaluation of its capabilities in handling dynamic environments.

Data availability statement

The data cannot be made publicly available upon publication due to legal restrictions preventing unrestricted public distribution. The data that support the findings of this study are available upon reasonable request from the authors.

Acknowledgments

This research was supported by the U.S. Department of Energy, through the ‘Data-Driven Decision Control for Complex Systems (DnC2S)’ project under the Advanced Scientific Computing Research program and the ‘Next Generation Compact SRF Accelerators for Industrial Applications’ project under the Nuclear Physics program. Contributions from Auralee Edelen and Ryan Roussel were supported by the Department of Energy, Laboratory Directed Research and Development program at SLAC National Accelerator Laboratory, under Contract DE-AC02-76SF00515. This manuscript has been authored by Jefferson Science Associates (JSA) operating the Thomas Jefferson National Accelerator Facility for the U.S. Department of Energy under Contract No. DE-AC05-06OR23177. The authors acknowledge support by the U.S. Department of Energy, Office of Science. The US government retains and the publisher, by accepting the article for publication, acknowledges that the US government retains a nonexclusive, paid-up, irrevocable, worldwide license to publish or reproduce the published form of this manuscript, or allow others to do so, for US government purposes. DOE will provide public access to these results of federally sponsored research in accordance with the DOE Public Access Plan (<http://energy.gov/downloads/doe-public-access-plan>).

Appendix. Benchmark with NSGA-II

In order to benchmark the pareto-front, we run the NSGA-II algorithm with varying population size and monitor the pareto-front coverage with the hypervolume. For simplicity, we fix the mutation and crossover probabilities to 0.01 and 0.95 chosen after a light hyper-parameter tuning. We run NSGA-II with population sizes of 128, 256, and 512 and observe that all the three variation of NSGA-II algorithm converges to similar solutions with small differences. In general, running with larger population size generally produce higher hypervolume for the same number of iterations. This is evident in figure A2, the figure shows hyper-volume coverage for NSGA-II evolution. We limit the number of generations to 50 K as the hyper-volume coverage plateaus. The hypervolume is slightly higher with increasing number of population size as each point contribute to the volume (area in this case). However, as seen in figure A1, the optimal pareto-front solutions with the three population sizes at 50 K generation is similar for smaller scale (8 and 16 -dimensional) environments, however, as we scale up the action and state space (32, and 200 -dimensional problems), a larger population size of 512 produce better optimal solutions. As such, we use NSGA-II with population size of 512 for comparison with other algorithms.

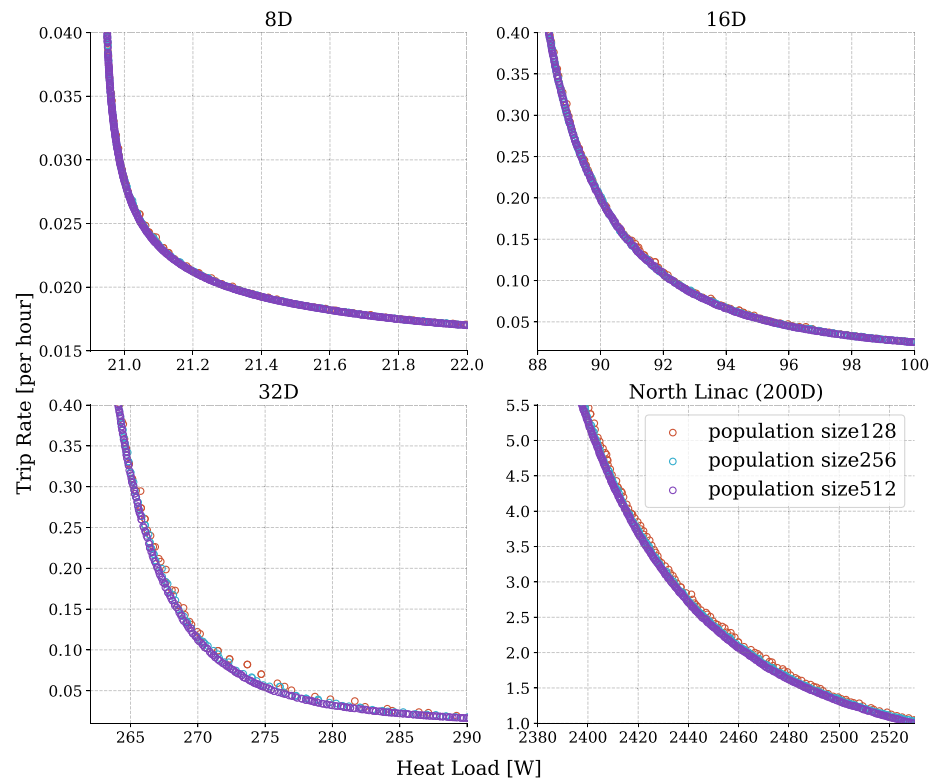


Figure A1. Comparison of Pareto-front obtained from NSGA-II with varying population sizes at 50 K generation.

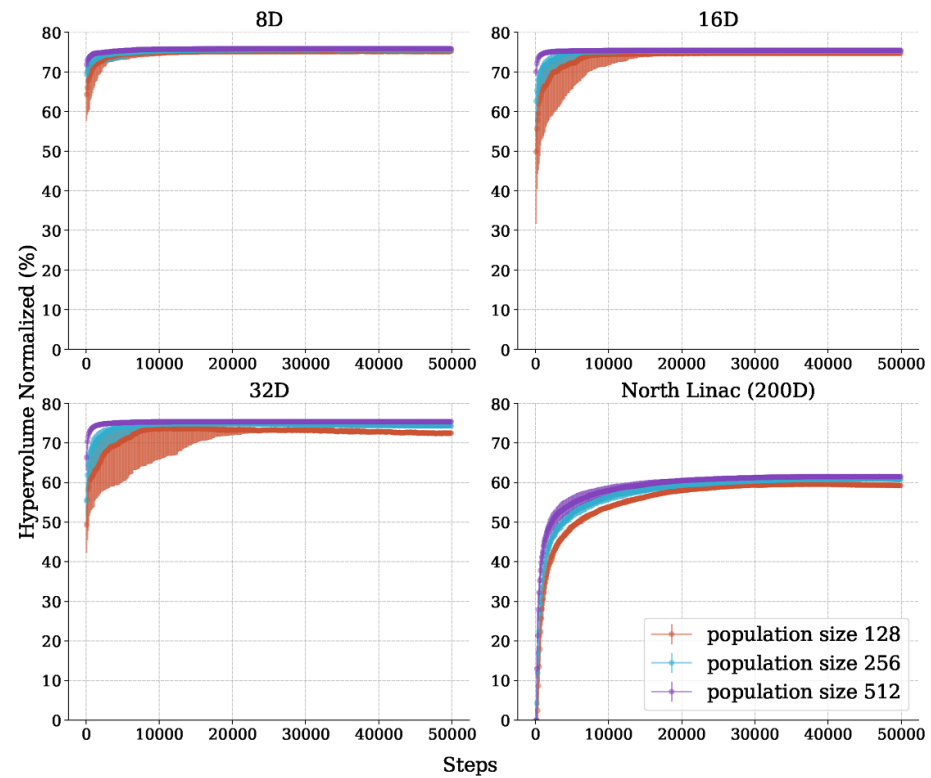


Figure A2. Hypervolume coverage of the pareto-front produced by NSGA-II during evolution. The area plots represents $2 - \sigma$ confidence bound around median over 16 trials with random initial seeds.

ORCID iDs

Kishansingh Rajput  <https://orcid.org/0000-0002-4430-9937>

Malachi Schram  <https://orcid.org/0000-0002-3475-2871>

Jonathan Colen  <https://orcid.org/0000-0003-4162-0276>

References

- [1] Adderley P A *et al* 2024 *Phys. Rev. Accel. Beams* **27** 084802
- [2] Fujimoto S, van Hoof H and Meger D 2018 Addressing function approximation error in actor-critic methods (arXiv:1802.09477)
- [3] Deb K, Pratap A, Agarwal S and Meyarivan T 2002 *IEEE Trans. Evol. Comput.* **6** 182–97
- [4] Roussel R, Hanuka A and Edelen A 2021 *Phys. Rev. Accel. Beams* **24** 062801
- [5] Jaisson T 2022 Deep differentiable reinforcement learning and optimal trading (arXiv:2112.02944)
- [6] Rajput K, Schram M, Blokland W, Alanazi Y, Ramuhalli P, Zhukov A, Peters C and Vilalta R 2024 *Mach. Learn.: Sci. Technol.* **5** 015044
- [7] Blokland W *et al* 2022 *Phys. Rev. Accel. Beams* **25** 122802
- [8] Edelen J P and Cook N M 2021 Anomaly detection in particle accelerators using autoencoders (arXiv:2112.07793 [physics])
- [9] Alanazi Y, Schram M, Rajput K, Goldenberg S, Vidyaratne L, Pappas C, Radaideh M I, Lu D, Ramuhalli P and Cousineau S 2023 *Mach. Learn. Appl.* **13** 100484
- [10] Marcato D, Arena G, Bortolato D, Gelain F, Martinelli V, Munaron E, Roetta M, Savarese G and Susto G A 2021 Machine learning-based anomaly detection for particle accelerators 2021 *IEEE Conf. on Control Technology and Applications (CCTA)* pp 240–6
- [11] Kafkes D and Schram M 2021 Developing robust digital twins and reinforcement learning for accelerator control systems at the fermilab booster 12th *Int. Particle Accelerator Conf.*
- [12] Schram M, Rajput K, Ns K S, Li P, St John J and Sharma H 2023 *Phys. Rev. Accel. Beams* **26** 602
- [13] Scheinker A, Pang X and Rybarczyk L 2013 *Phys. Rev. Spec. Top.* **16** 102803
- [14] Scheinker A, Hirlaender S, Velotti F M, Gessner S, Della Porta G Z, Kain V, Goddard B and Ramjiawan R 2020 *AIP Adv.* **10** 423
- [15] c v T B, Hofler A S, Reeves C J, Khan S A, Krafft G A, Benesch J, Freyberger A and Ranjan D 2014 *Phys. Rev. Spec. Top.* **17** 101003
- [16] Huber K, Wirtz T and Hoang H Q 2024 *Nucl. Instrum. Methods Phys. Res. A* **1067** 169702
- [17] Neveu N, Spentzouris L, Adelmann A, Ineichen Y, Kolano A, Metzger-Kraus C, Bekas C, Curioni A and Arbenz P 2019 *Phys. Rev. Accel. Beams* **22** 054602
- [18] Edelen A, Neveu N, Frey M, Huber Y, Mayes C and Adelmann A 2020 *Phys. Rev. Accel. Beams* **23** 044601
- [19] Kranjčević M 2021 *Phys. Rev. Accel. Beams* **24** 014601
- [20] Boltz T, Martinez J L, Xu C, Baker K R, Roussel R, Ratner D, Mustapha B and Edelen A L 2024 arXiv:2403.03225
- [21] Hanuka A, Huang X, Shtalenkova J, Kennedy D, Edelen A, Zhang Z, Lalchand V R, Ratner D and Duris J 2021 *Phys. Rev. Accel. Beams* **24** 072802
- [22] Roussel R *et al* 2024 *Phys. Rev. Accel. Beams* **27** 084801
- [23] Kuklev N, Borland M, Fystro G, Shang H and Sun Y 2022 *Proc. NAPAC* **22** 842
- [24] Ji F *et al* 2024 *Nat. Commun.* **15** 4726
- [25] Kirschner J, Nonnenmacher M, Mutny M, Krause A, Hiller N, Ischebeck R and Adelmann A 2019 Bayesian optimisation for fast and safe parameter tuning of swissfel FEL2019 *Proc. 39th Int. Free-Electron Laser Conf. (JACoW Publishing)* pp 707–10
- [26] Ferran P A *et al* 2023 *Phys. Rev. Accel. Beams* **26** 084601
- [27] St J J *et al* 2021 *Phys. Rev. Accel. Beams* **24** 104601
- [28] Kaiser J *et al* 2024 *Sci. Rep.* **14** 15733
- [29] Scomparin L *et al* 2024 Preliminary results on the reinforcement learning-based control of the microbunching instability *Proc. IPAC'24 (Int. Particle Accelerator Conf. no 15)* (JACoW Publishing) pp 1808–11
- [30] Hirlaender S, Lamminger L, Pochaba S, Santamaria Garcia A, Xu C, Scomparin L, Kaiser J and Kain V 2024 Towards few-shot reinforcement learning in particle accelerator control 15th *Int. Particle Accelerator Conf., (Nashville, Tennessee, 19–24 May 2024)* (Proc.) ed F Pilat (JACoW Publishing) pp 1804–7
- [31] Vidyaratne L, Carpenter A, Powers T, Tennant C, Ifteharuddin K, Rahman M M and Shabalina A 2022 *Front. Artif. Intell.* **4** 718950
- [32] Benesch J 2015 A longitudinal study of field emission in CEBAF's SRF cavities 1995–2015 (arXiv:1502.06877)
- [33] Felten F, Alegre L N, Nowé A, Bazzan A L C, Talbi E G, Danoy G and Silva B C 2023 A toolkit for reliable benchmarking and research in multi-objective reinforcement learning *Proc. 37th Conf. on Neural Information Processing Systems (NeurIPS 2023)*
- [34] Abadi M *et al* 2015 TensorFlow: large-scale machine learning on heterogeneous systems software (available at: www.tensorflow.org/)
- [35] Guerreiro A P, Fonseca C M and Paquete L 2021 *ACM Comput. Surv.* **54** 1–42
- [36] Blank J and Deb K 2020 *IEEE Access* **8** 89497–509
- [37] Rasmussen C E and Williams C K I 2006 *Gaussian Processes for Machine Learning* (The MIT Press)
- [38] Duris J *et al* 2020 *Phys. Rev. Lett.* **124** 124801
- [39] Goan E and Fookes C 2020 *Bayesian Neural Networks: An Introduction and Survey* (Springer) pp 45–87
- [40] Eriksson D, Pearce M, Gardner J R, Turner R and Poloczek M 2019 *Scalable Global Optimization via Local Bayesian Optimization* (Curran Associates Inc.)
- [41] Pardalos P, Zhigljavsky A and Žilinskas J 2016 *Advances in Stochastic and Deterministic Global Optimization* vol 107
- [42] Daulton S, Balandat M and Bakshy E 2021 Parallel bayesian optimization of multiple noisy objectives with expected hypervolume improvement (arXiv:2105.08195)
- [43] Roussel R, Edelen A, Bartnik A and Mayes C 2023 Xopt: a simplified framework for optimization of accelerator problems using advanced algorithms *Proc. IPAC'23 (IPAC'23 -14th Int. Particle Accelerator Conf. no 14)* (JACoW Publishing) pp 4796–9
- [44] Balandat M, Karrer B, Jiang D R, Daulton S, Letham B, Wilson A G and Bakshy E 2020 BoTorch: a framework for efficient Monte-Carlo Bayesian optimization *Advances in Neural Information Processing Systems* vol 33
- [45] Zitzler E, Deb K and Thiele L 2000 *Evol. Comput.* **8** 173–95
- [46] Hayes C F *et al* 2022 *Auto. Agents Multi-Agent Syst.* **36** 26

- [47] Basaklar T, Gumussoy S and Ogras U 2023 PD-MORL: preference-driven multi-objective reinforcement learning algorithm *The 11th Int. Conf. on Learning Representations* (<https://openreview.net/forum?id=zS9sRyaPFIJ>)
- [48] Hufe B and Savine A 2020 Differential machine learning (arXiv:2005.02347)
- [49] Snoek J, Larochelle H and Adams R P 2012 Practical Bayesian optimization of machine learning algorithms *Advances in Neural Information Processing Systems* vol 25, ed F Pereira, C Burges, L Bottou and K Weinberger (Curran Associates, Inc.)
- [50] Kaiser J, Xu C, Eichler A and Santamaria Garcia A 2024 *Phys. Rev. Accel. Beams* **27** 054601
- [51] Gonzalez-Aguilera J P, Kim Y K, Roussel R, Edelen A and Mayes C 2023 *JACoW* vol IPAC2023 p WEA065